

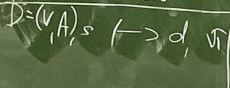
Breitensuche - initialisieren (S):
 ▷ Initialisieren der BFS für $D=(V,A)$ und Startknoten S.

1. for $u \in V \setminus \{S\}$:
2. Farbe[u] = Weiss
3. $d[u] := \infty$ ▷ Abstand zu u in Aufbau.
4. $\pi[u] = NIL$ ▷ von welchem Knoten aus wurde u erreicht?
 NIL = von keinem (noch) Vorgänger von u
5. end
6. Farbe[S] = Grau
7. $d[S] := 0$
8. $\pi[S] := NIL$
9. Pushiere eine leere Warteschlange Q mit max. Kapazität |V|.
10. Enqueue(Q, S)

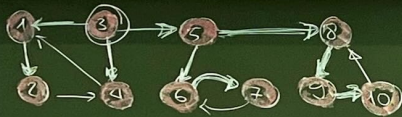
Breitensuche (S):

1. Breitensuche - initialisieren(S)
2. while Q nicht leer:
3. $u = \text{Dequeue}(Q)$
4. for $v \in N[u]$:
5. if Farbe[v] = Weiss:
6. Farbe[v] = Grau
7. Enqueue(Q, v)
8. $d[v] = d[u] + 1$
9. $\pi[v] = u$
10. end
11. Farbe[u] = Schwarz
12. end
13. end

Ziel: einen kürzesten Weg von S zu jedem erreichbaren Knoten
 von S aus



3.12 Bsp.



Breitensuche (3)

- Grau
- Schwarz

Q: 3

1 4 5
 4 5 2
 5 2
 2

6 8
 6 8
 7 9
 9

10

Abgewandelte Aufgabe:

man darf sowohl ≥ 3 als auch von 10 zu anderen Knoten kommen.

Möglichkeit 1

2x Breitensuche (S)
 mit $S=3$ und $S=10$
 und dann jeweils die bessere Variante nehmen

Möglichkeit 2:

Initialisierung ändern
 $Q = [3, 10]$
 $d[3] = d[10] = 0$

Möglichkeit 3

Man ändert $D=(V,A)$, indem man $3 \rightarrow 10$ setzt, und dann die BFS

$d[5] = 0$ $\pi[5] = NIL$
 $d[1] = d[4] = d[5] = 1$ $\pi[1] = \pi[4] = \pi[5] = 5$
 $d[2] = 2$, $\pi[2] = 1$
 $d[6] = d[8] = 2$, $\pi[6] = \pi[8] = 5$
 $d[7] = 3$, $\pi[7] = 6$
 $d[9] = 3$, $\pi[9] = 8$
 $d[10] = 4$, $\pi[10] = 9$

(2)

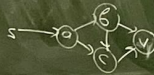
3.13. Wir reden bei der Analyse zuerst über die Laufzeit und dann über die Korrektheit.

(4)

3.14. Theorem. Die Laufzeit des Breiten suches ist $O(|V| + |A|)$ (auf einem Digraphen $D = (V, A)$).

Beweis: Der Algorithmus ist so aufgebaut, dass für jeden Knoten nur die folgenden Farbumkehrungen möglich sind: Weiss zu grau, grau zu schwarz. Ein Knoten kann nur dann in Q aufgenommen werden, wenn seiner Farbe weiss ist, dabei wird der Knoten bei der Aufnahme in Q grau gefärbt. Also wird jeder Knoten höchstens einmal in Q aufgenommen. Der Aufwand pro Knoten ist: einmal in Q aufnehmen, einmal aus Q entfernen, alle Nachbarn aus $N(u)$ scannen. Also ist der Gesamtaufwand $O(|V|) + O(\sum_{u \in V} |N(u)|) = O(|V| + |A|)$ $\square$

3.15. Def. Wir nennen eine (s, v) -Pfad der Länge $d(s, v)$ einen kürzesten (s, v) -Pfad.



3.16 Lemma Gilt $(u, v) \in A$,
so ist
 $d(s, v) \leq d(s, u) + 1$.

Beweis: klar.

